

CEN 116- EXERCISE

1) Write the output of the following program. **24 0 30**

```
#include <stdio.h>
void calculate(int *px, int *py) {
    int *pz;
    static int k = 0;
    for (pz = py; pz < py + 4; pz++) {
        *pz = *(py+k) + *(px+k);
        k++; }
}
int main() {
    int a[4][4]={0,1,2,3},{4,5,6,7},{8,9,10,11},{12,13,14,15}};
    int (*px)[4], *py[8];
    py[0] = a[3]; py[1] = a[0];
    px = &a[3];
    calculate(px[0],py[0]);
    printf("%d %d %d\n",*py[0],*py[1],(*py)[3]);
    return 0;
}
```

2) Write the output of the following program. **Variable d was not initialized, it must be initialized. However, for every value in the range of integer produces the same result: 144--14**

```
#include <stdio.h>

int func (int k, int *px)
{
    *px = *px & k ^ 10;

    return *px;
}
int main()
{
    int a = 4, b = 8, c,d;

    c = a >> 2;
    c += b;
    c = c << 4;

    d = a >> 2 + (a << 4) & 10 ^ func(4,&d) | 4;

    printf("%d--%d\n",c,d);

    return 0;
}
```

- 3) Write a C program to find the unique names in a file. Your program will create a binary file to keep ids, names and salaries of employees. You will use structs to write each record to the file. After creation of the file you will find unique names in that file. You will print only names.

EX: 123 John 1500
234 Jersey 2300
345 John 1200
290 George 1890
190 John 1700 you will display : John Jersey George

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    int id;
    char name[100];
    char surname[100];
    float salary;
}EMPLOYEE;

int insert (EMPLOYEE *a)
{
    static int count = 0;

    count++;

    printf("ENTER ID:\t");
    scanf("%d",&a->id);
    printf("ENTER NAME :\t");
    scanf("%s",a->name);
    printf("ENTER SURNAME:\t");
    scanf("%s",a->surname);
    printf("ENTER SALARY\t");
    scanf("%f",&a->salary);
    return count;
}

int main ()
{
    FILE * ft;
    EMPLOYEE emps;
    char ch;
    int count = 0;
    int success, i,j = 0, unmatched = 0;
    char ** list;

    ft = fopen("records.txt","w+b");
    if(!ft)
        exit(-1);
```

```

while(1)
{
    printf("TO INSERT PRESS y, TO LIST press n");
    scanf(" %c",&ch);
    if(ch == 'n')
        break;
    count = insert(&emps);
    success = fwrite(&emps,sizeof(emps),1,ft);
    if(success != 1)
        exit(-1);
}
list = (char **) calloc(count,sizeof(char*));
for (i = 0; i<count;i++)
{
    list[i] = (char *) calloc(100,sizeof(char));
}

printf("LISTING UNIQUE NAMES\n");
rewind(ft);
while(fread(&emps,sizeof(emps),1,ft)!=0)
{
    unmatched = 0;
    for (i = 0; i < j; i++)
    {
        if(strcmp(emps.name,list[i]) == 0)
        {
            unmatched = -1;
            break;
        }
        else
        {
            unmatched++;
        }
    }
    if(unmatched >=0)
    {
        strcpy(list[j],emps.name);
        printf("%s\n",emps.name);
        j++;
    }
}
for (i = 0; i<count;i++)
{
    free(list[i]);
}
free(list);

fclose(ft);
return 0;
}

```

- 4) Write a C program including a macro to PRINT variable name and value when it is called. You will use that macro to debug your code. You have to control whether you are using it in DEBUG mode or not.

```
#define DEBUGS 1
#define PRINT_VAL(a) \
printf("At line %d --", __LINE__); \
printf("#a " contains: %d\n", (a))
#include <stdio.h>

int func (int *px, int *py)
{
    #if DEBUGS
        PRINT_VAL(*(px+1));
        PRINT_VAL(*(py+1));
        PRINT_VAL(*(px+1) * *(py+1));
    #endif

    return *(px+1) * *(py+1);
}

int main (void)
{
    int a[]={0,4,6,7}, b[]={1,6,8,9};

    #if DEBUGS
        PRINT_VAL(func(&a[1],&b[2]));
    #endif
}
```

- 5) Write a C program to keep people and company information in a linked list. (You will use unions)

```
#include <stdio.h>
#include <stdlib.h>

typedef struct
{
    char name[50];
    char surname[50];

}PERSON;

typedef union
{
    PERSON prsn;
    char company[50];
}INFO;

typedef struct LLIST
{
    char ch;
```

```

    INFO inf;
    struct LLIST *next;
}BOOK;

BOOK *first =NULL, *last =NULL;

void insert(BOOK *listt)
{
    char pc;
    printf("COMPANY c OR PERSON p");
    scanf(" %c", &pc);
    if(pc == 'p')
    {
        listt->ch = 'p';
        printf("ENTER NAME AND SURNAME\n");
        scanf(" %s", listt->inf.prsn.name);
        scanf(" %s", listt->inf.prsn.surname);

    }
    else
    {
        listt->ch = 'c';
        printf("ENTER COMPANY NAME\n");
        scanf(" %s", listt->inf.company);
    }

    if(first == NULL)
    {
        first = listt;
        last = first;
        last->next = NULL;
    }
    else
    {
        last->next = listt;
        last = listt;
        last->next = NULL;
    }
}

int main()
{
    BOOK *list = NULL, *p =NULL;
    char ch;
    int count = 0;
    while(1)
    {
        printf("TO INSERT PRESS y, TO LIST press n");
        scanf(" %c",&ch);
        if(ch == 'n')
            break;
        count++;
    }
}

```

```

        list = (BOOK*) malloc(sizeof(BOOK));
        insert(list);
    }
    printf("LISTING\n");
    p = first;
    while (p)
    {
        if(p->ch == 'p')
        {
            printf("%s--%s\n", p->inf.prsn.name, p->inf.prsn.surname);
        }
        else
        {
            printf("%s\n", p->inf.company);
        }
        p = p->next;
    }

    free(list);

    return 0;
}

```

- 6) Write a C program to print entered string in reverse order. Your program must include **recursive function** and **char pointer**.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char* reverse(char * str, char *rstr)
{
    if(*str == '\0')
    {
        rstr--;
        return rstr;
    }

    *rstr = *str;
    str--;
    rstr++;
    return reverse(str, rstr);
}

int main ()
{
    char *str, *rstr;

```

```

    unsigned long int nlen;

    str = (char *) calloc(10,sizeof(char));
    rstr = (char*) calloc(10,sizeof(char));
    *str = '\0';
    str++;
    printf("Enter a string:\n");
    scanf("%8s",str);
    nlen = strlen(str);

    str += strlen(str)-1;
    reverse(str,rstr);

    printf("%s\n",rstr);
    str -= nlen;
    free(rstr);
    free(str);

    return 0;
}

```

- 7) Write a C program to change the id values of the repeated names to 333 in file created for QUE 3.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {

    int id;
    char name[100];
    char surname[100];
    float salary;
}EMPLOYEE;

int main ()
{
    FILE *ft;
    EMPLOYEE emps;
    char **list =NULL;
    int unmatched,i,j=0,count = 0;

    ft = fopen("records.txt","r+b");

    while(fread(&emps,sizeof(EMPLOYEE),1,ft))
    {
        count++;
    }
}

```

```

list = (char **) calloc(count, sizeof(char*));
for (i = 0; i < count; i++)
{
    list[i] = (char *) calloc(100, sizeof(char));
}
fseek(ft, 0, SEEK_SET);

while(fread(&emps, sizeof(EMPLOYEE), 1, ft))
{

    unmatched = 0;
    for (i = 0; i < j; i++)
    {
        if(strcmp(emps.name, list[i]) == 0)
        {

            unmatched = -1;
            break;

        }
        else
        {
            unmatched++;
        }
    }
    if(unmatched >= 0)
    {

        strcpy(list[j], emps.name);
        j++;
    }
    else
    {

        fseek(ft, -1 * sizeof(emps), SEEK_CUR);
        emps.id = 333;
        fwrite(&emps, sizeof(emps), 1, ft);
    }

}
rewind(ft);
printf("LISTING\n");

while(fread(&emps, sizeof(EMPLOYEE), 1, ft))
{
    printf("%d\t %s\t %s\t %f\n", emps.id, emps.name, emps.surname, emps.salary);
}

for (i = 0; i < count; i++)
{
    free(list[i]);
}

```

```

    }
    free(list);

    fclose(ft);
    return 0;
}

```

- 8) Write 2 header files containing the same function declarations with different operations and write a C program and try to include these header files and if you get errors, try to solve it by using MACROS.

program.c

```

#define FLOATING 1

#if FLOATING
#include "func2.h"
#else
#include "func1.h"
#endif

#if FLOATING
#define PRINT(x)\
printf("%f\n",x)
#else
#define PRINT(x)\
printf("%d\n",x)
#endif

#include <stdio.h>

int main()
{
    int a = 5, b = 10;
    float c = 4, d = 2.6;
    PRINT(adding(a,b));
    PRINT(multiply(c,d));
    return 0;
}

```

func1.h

```

#ifndef func1_h
#define func1_h
int adding(int a, int b)
{
    return a+b;
}

int multiply(int a, int b)
{
    return a*b;
}

#endif /* func1_h */

```

func2.h

```
#ifndef func2_h
#define func2_h

float adding(float a, float b)
{
    return a+b;
}

float multiply(float a, float b)
{
    return a*b;
}

#endif /* func2_h */
```